1. Legacy-to-Modern Software Modernization Projects

Focus: legacy-system analysis, architecture assessment, transformation planning, testing, performance analysis, and final evaluation.

1.1 IBM CICS General Insurance Application (GenApp)

Main objective: Analyze a traditional mainframe insurance application that manages customers and insurance policies, and migrate its core functionality to .NET. The application's separation into presentation, business, and data-management layers makes it suitable for studying how legacy layering maps to a modern architecture.

Topic / Technology: COBOL, CICS, BMS, Db2, VSAM; migration to .NET (ASP.NET Core) and a relational database.

Implementation note: The scope is the COBOL application in the repository's base component. Since a CICS/Db2 environment will generally not be available, the legacy code is analyzed statically; behavioral equivalence is demonstrated through tests derived from the COBOL logic.

Repository: cicsdev/cics-genapp

1.2 AWS CardDemo

Main objective: Study a realistic mainframe credit-card management system, identify legacy architecture and coding characteristics, assess migration options, and migrate selected functionality to .NET. The project is well suited to discovery, migration assessment, service extraction, and performance comparison.

Topic / Technology: COBOL, CICS, VSAM, JCL, RACF; mainframe modernization and service decomposition; migration to .NET (ASP.NET Core) and a relational database.

Implementation note: The application contains varied coding styles, which makes it suitable for legacy-code analysis and transformation planning. The target platform is .NET. Publicly available third-party ports of CardDemo to other platforms (e.g., Java/Spring Boot) may not be used as the basis of the implementation.

Repository: aws-samples/aws-mainframe-modernization-carddemo

1.3 Microsoft eShopModernizing

Main objective: Analyze a traditional .NET Framework application and migrate it to a modern, currently supported .NET version, including moving the legacy web layer (ASP.NET MVC or Web Forms) to ASP.NET Core (e.g., Razor Pages or Blazor), and evaluate the result using modern deployment and architecture practices.

Topic / Technology: .NET Framework, ASP.NET MVC/Web Forms, modern .NET (e.g., .NET 10 LTS), ASP.NET Core, containers, application modernization, architecture migration.

Repository: dotnet-architecture/eShopModernizing

3. Attack-Graph-Based Security Architecture Projects

Focus: vulnerability assessment, severity/exploitability analysis, attack-path modeling, attack graphs, and defense-in-depth security architecture.

3.1 AI-Assisted Attack Graph Generation for OWASP Juice Shop

Main objective: Analyze vulnerabilities in OWASP Juice Shop, classify findings using established security knowledge bases, construct attack paths and an attack graph, and evaluate how security controls such as WAF, IAM, and IDS/IPS change the attack graph and residual risk.

Topic / Technology: Node.js/Express/Angular, web security, CWE, CAPEC, MITRE ATT&CK, attack graphs, defense-in-depth.

Repository: juice-shop/juice-shop

3.2 ATT&CK-Based Security Architecture for OWASP WebGoat

Main objective: Analyze WebGoat vulnerabilities, classify them using CWE/CAPEC, associate relevant adversarial techniques where appropriate, build attack paths, and design a layered security architecture to reduce the identified risks.

Topic / Technology: Java, Spring, web security, CWE/CAPEC, MITRE ATT&CK/STIX, attack-path analysis.

Repository: WebGoat/WebGoat

4. AI-Assisted Discovery of Software Vulnerabilities

Focus: attack-surface analysis, static and dynamic analysis, fuzzing, dependency analysis, AI-assisted code review, candidate vulnerability verification, severity assessment, and regression testing.

4.1 AI-Assisted Vulnerability Discovery in DVWA

Main objective: Analyze both source code and the running application to identify, verify, and assess documented and potentially undocumented candidate vulnerabilities. Students should distinguish confirmed findings from false positives and evaluate severity.

Topic / Technology: PHP, MariaDB, SAST, DAST, fuzzing, dependency analysis, AI-assisted code review.

Repository: digininja/DVWA

4.2 AI-Assisted Vulnerability Discovery in OWASP Juice Shop

Main objective: Analyze the frontend, backend, and REST API attack surface of a modern web application; combine automated analysis with AI-assisted source-code review; identify candidate vulnerabilities; verify them in an isolated environment; and separate true findings from false positives.

Topic / Technology: Node.js, Express, Angular, REST API, SAST, DAST, fuzzing, dependency analysis.

Repository: juice-shop/juice-shop

4.3 AI-Assisted REST API Vulnerability Discovery

Main objective: Analyze REST API endpoints using static analysis, dynamic analysis, and fuzzing; use AI-assisted source-code analysis to investigate suspicious code; validate candidate vulnerabilities; apply or propose fixes; and perform regression testing.

Topic / Technology: Python, FastAPI, PostgreSQL, Docker, REST API, SAST/DAST, fuzzing.

Repository: theowni/Damn-Vulnerable-RESTaurant-API-Game

4.4 AI-Assisted Source-Code Vulnerability Discovery in NodeGoat

Main objective: Perform source-code-centered vulnerability discovery using SAST and AI-assisted review, then validate candidate findings against the running application. Students should compare tool findings, identify false positives, assess severity, and verify fixes through regression tests.

Topic / Technology: Node.js, Express, MongoDB, SAST, AI-assisted code review, DAST, regression testing.

Repository: OWASP/NodeGoat

5. Open-Source Software Enhancement and Contribution Projects

Focus: understanding an actively maintained open-source codebase, identifying an improvement opportunity, designing and implementing an enhancement, testing and documenting it, and submitting the work through the project's contribution process.

5.1 Excalidraw — Drawing Application Enhancement

Main objective: Develop an improvement to the editor, export workflow, keyboard interaction, accessibility, or drawing functionality while following the project's contribution and testing practices.

Topic / Technology: TypeScript, React, interactive editor, accessibility, automated testing.

Repository: excalidraw/excalidraw

5.2 Joplin — Note-Taking Application Enhancement

Main objective: Improve an existing feature such as search/filtering, editing, tagging, import/export, or usability and deliver the change with tests, documentation, and an open-source contribution workflow.

Topic / Technology: TypeScript, React, desktop/mobile application development, search/editor features, testing.

Repository: laurent22/joplin

5.3 Memos — Knowledge/Note Management Enhancement

Main objective: Implement a meaningful enhancement such as advanced search/filtering, tagging, API behavior, or user-interface functionality and validate it across the relevant frontend/backend components.

Topic / Technology: Go, React, API development, full-stack engineering, testing.

Repository: usememos/memos

5.4 BookStack — Content Management Enhancement

Main objective: Enhance a content-management feature such as search, permissions, content organization, export, or user experience while respecting the project's architecture, coding standards, testing practices, and contribution guidelines.

Topic / Technology: PHP, Laravel, relational database, web application engineering, testing.

Repository: BookStackApp/BookStack